

The universal construction Co-Product

Luca Roversi

Learning outcomes

- What is a Co-Product?
- Pattern of the Co-Product
- Why the Co-Product pattern is the best one?
- Why other Co-Product candidate pattern are worst?

Cartesian Co-Product in a category $\mathcal{C}$

A Co-Product in a category $\mathcal{C}$ is a triple $(A + B, \iota_l, \iota_r)$ such that:

- A, B are in $\text{obj}(\mathcal{C})$
- ι_l is in $\mathcal{C}(A, A + B)$, and ι_r is in $\mathcal{C}(B, A + B)$
- for every C in $\text{obj}(\mathcal{C})$, a *unique* s in $\mathcal{C}(A + B, C)$ exists s.t.:

$$\begin{array}{ccccc} A & \xrightarrow{\iota_l} & A + B & \xleftarrow{\iota_r} & B \\ & \searrow f & \downarrow s & \swarrow g & \\ & & C & & \end{array}$$

commutes, i.e.:

- $f = s \circ \iota_l$
- $g = s \circ \iota_r$

Cartesian Co-Product in Set (via Haskell syntax)

The pattern of every Co-Product:

- $A + B$ has labeled elements to record their origin
- *Injections* connecting A, B to $A + B$

Haskell original:

```
ghci> :t Left
Left :: a -> Either a b           -- in_l
ghci> :t Right
Right :: b -> Either a b         -- in_r
ghci> :i Either
type Either :: * -> * -> *
data Either a b = Left a | Right b -- a + b
```

Hand-made Co-product in Haskell:

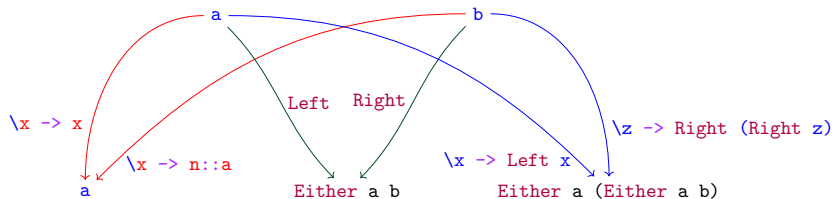
```
ghci> :{
ghci|     data CoP a b where
ghci|         inl :: a -> CoP a b
ghci|         inr :: b -> CoP a b
ghci| :}
```

Why **Either** a b is the Co-Product of a and b ?

Could we use:

- simply a or
- the more complex **Either** (**Either** a b)

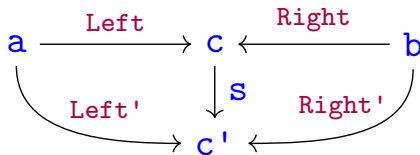
as Co-Product of a and b ?



Criterion to characterize the Co-Product of a and b

For every Co-Product *candidate* $a \xrightarrow{\text{Left}'} c' \xleftarrow{\text{Right}'} b$ of a, b ,
the actual co-product c of a and b must be *better* than c' :

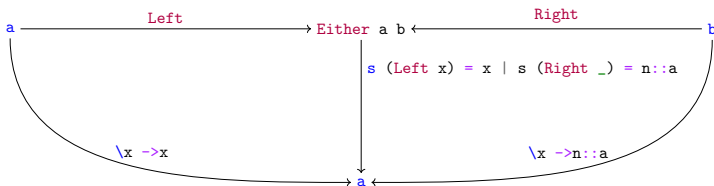
- **there exists a unique** arrow $c \xrightarrow{s} c'$
- s.t. the injections $\text{Left}, \text{Right}$ are better than $\text{Left}', \text{Right}'$:



commutes, i.e.:

- $\text{Left}' = s \circ \text{Left}$, i.e. Left' is *reconstructed* by s and Left
- $\text{Right}' = s \circ \text{Right}$, i.e. Right' is *reconstructed* by s and Right
- or, equivalently:
 - s factorizes Left' and Right' using Left and Right

(Either, Left, Right) better than other proposals



(`Either`, `Left`, `Right`) better than (`a`, `\x -> x`, `\x -> n::a`) because:

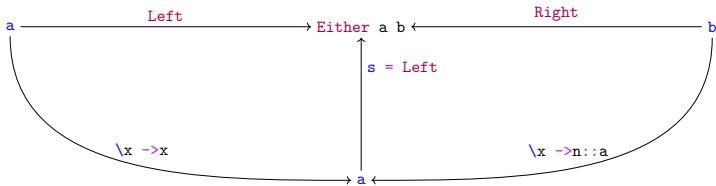
`s (Left x) = x`

`s (Right _) = n::a`

factorizes `\x -> x` and `\x -> n::a`:

- `s . Left == \x -> x`
- `s . Right == \x -> n::a`

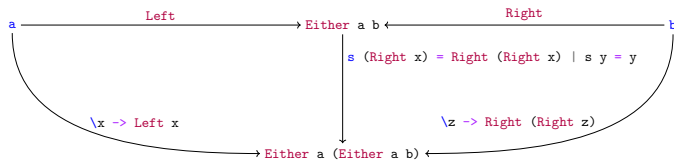
$(a, \lambda x. x, \lambda x. n :: a)$ worst than
 $(\text{Either}, \text{Left}, \text{Right})$



`s = Left` is one of the two reasonable choices:

- its output matches the type
- **it can** factorize `Left`
- **but it cannot** factorize `Right`

`(Either, Left, Right)` better than other proposals



`(Either, Left, Right)` better than

`(Either a (Either a b), \x -> Left x, \z -> Right (Right z))`

because:

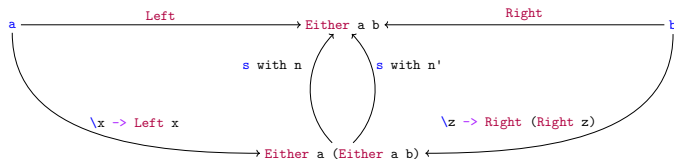
`s (Right x) = Right (Right x)`

`s y = y`

factorizes `\x -> Left x, \z -> Right (Right z)`:

- `s (Left x) == Left x`
- `s (Right z) == Right (Right z)`

$(\text{E } a \text{ (E } a \text{ b) , } \backslash x \rightarrow \text{L } x \text{ , } \backslash z \rightarrow \text{R } (\text{R } z))$
 worst than $(\text{Either, Left, Right})$



The factorization **is not unique**:

$s \text{ (Left } x) = \text{Left } x$

$s \text{ (Right } y) = s' \ y$

where

$s' \text{ (Right } z) = \text{Right } z$

$s' \text{ (Left } _) = \text{Left } (n :: a)$

factorizes `Right` for any `n :: a`

Summary and Exercises

Universal constructions as “best fitting” patterns

- (Cartesian) Co-Product.

The universal definition says when an object A is “more a co-product” than any other object B

Unicity and Isomorphism

- Is the Co-Product of `a` and `b` unique up to isomorphism?
- Are `Either a Void` and `a` isomorphic?
Notice. `import Data.Void` required to use `Void`,
`absurd :: Void -> a`
- Are `(a, Either b c)` and `Either (a, b) (a, c)` isomorphic?
- Are `Either a a` and `(Either () ())`, `a` isomorphic?

Co-Product in Haskell

Are the following triples:

- `(Either (Either Bool Int) Int , \x -> Left (Right x)
 , \z -> Left (Left z))`
- `(Either Int Bool , \x -> Left (x + 1) , \y -> Right y )`
- `(Either Int Bool , \x -> Left x , \y -> Right (not y) )`
- `(Int , \x :: Int -> x , \y :: Bool -> if y then 0 else 1 )`
- `(Int , \x :: Int -> if x < 0 then x else x+2 ,
 \y :: Bool -> if y then 0 else 1)`

Co-Products between `Int` and `Bool`?



Milewski, B. and I. Tabachnik (2019). *Category Theory for Programmers*.

<https://bartoszmilewski.com/2014/10/28/category-theory-for-programmers-the-preface/>. Lightning Source UK. ISBN: 9780464243878. URL:

<https://books.google.it/books?id=hA55yQEACAAJ>.